

Python Programming

Practice 7 — Week 7: Advanced OOP

Academic Year 2025–2026

Topics Covered

Task	Topic	Concepts
1	Inheritance	Base class, __init__, __str__, analyse()
2	Inheritance	Child class, super().__init__(), method overriding
3	Method Overriding	Override print_results(), call super().print_results()
4	Association	USES-A relationship, class Report
5	Polymorphism	Same interface, different behaviour per class

Overview

This practice extends the class structure from Practice 6. You will turn DataAnalyser into a base class, create a variant-specific child class, demonstrate method overriding, association, and polymorphism.

Variant Assignment

Task 1, 3, 4, and 5 are the same for all variants. Task 2 depends on your variant — create the child class for your analysis type.

Last digit of ID	Variant	Child Class for Task 2
1, 2, 3	A	class GpaAnalyser(DataAnalyser)
4, 5, 6	B	class CountryAnalyser(DataAnalyser)
7, 8	C	class SleepAnalyser(DataAnalyser)
9, 0	D	class TopStudentsAnalyser(DataAnalyser)

Submission & Defense

File to submit	main.py
Platform	LMS — lms.astanait.edu.kz
Deadline	Before Week 8 practice session
Defense	Week 8 — bring your laptop. Instructor removes key parts; you have 5 minutes to restore the logic.

■ Late submissions are not accepted. Submitting another student's variant will be treated as academic misconduct.

Tasks 1 & 2

Task 1 — Base Class DataAnalyser (All Variants)

Refactor the DataAnalyser class from Practice 6 into a proper base class. It will define the shared structure that all child classes inherit.

Technical requirements:

- ✓ Define class DataAnalyser
- ✓ Define `__init__(self, students)`:
 - Store students as `self.students`
 - Set `self.result = {}`
- ✓ Define method `analyse(self)`:
 - Print "Not implemented – use a child class"
 - This method will be overridden in every child class
- ✓ Define method `print_results(self)`:
 - Loop over `self.result` and print each key-value pair
- ✓ Define `__str__(self)`:
 - Return a string: "DataAnalyser: base class, {len(self.students)} students"

Console:

```
analyser = DataAnalyser(students)
print(analyser)

DataAnalyser: base class, 10000 students

analyser.analyse()
Not implemented – use a child class
```

Task 2 — Child Class — Your Variant

Create a child class that inherits from DataAnalyser and overrides `analyse()`. Use the analysis logic from your variant in Practices 4 and 5.

Technical requirements:

- ✓ Define your child class inheriting from DataAnalyser:

Variant	Class definition
A	<code>class GpaAnalyser(DataAnalyser)</code>
B	<code>class CountryAnalyser(DataAnalyser)</code>
C	<code>class SleepAnalyser(DataAnalyser)</code>
D	<code>class TopStudentsAnalyser(DataAnalyser)</code>

- ✓ Define `__init__(self, students)`:
 - Call `super().__init__(students)` to initialise the parent class
- ✓ Override `analyse(self)`:
 - Insert your variant's analysis logic from Practice 5
 - Store the result dictionary in `self.result`
- ✓ Override `__str__(self)`:

- Return a string describing your child class and student count

Console (Variant A example):

```
analyser = GpaAnalyser(students)
print(analyser)

GpaAnalyser: GPA Statistics, 10000 students

analyser.analyse()
analyser.print_results()

-----
GPA Analysis
-----
Total students : 10000
Average GPA : 3.45
Highest GPA : 4.0
Lowest GPA : 1.0
Students GPA>3.5 : 6028
-----
```

Tasks 3 & 4

Task 3 — Method Overriding — `print_results()` (All Variants)

Override `print_results()` in your child class to produce a more detailed, formatted output than the base class version.

Technical requirements:

- ✓ In the base class `DataAnalyser`, `print_results()` simply loops over `self.result` and prints `key: value`
- ✓ In your child class, override `print_results(self)`:
 - Print a formatted header with your analysis name
 - Call `super().print_results()` to print the base output
 - Print a formatted footer
- ✓ The overridden method must produce noticeably different output than the base class version

Base class output:

```
total_students: 10000
average_gpa: 3.45
max_gpa: 4.0
min_gpa: 1.0
high_performers: 6028
```

Child class output (Variant A example):

```
=====
GPA ANALYSIS REPORT
=====
total_students: 10000
average_gpa: 3.45
max_gpa: 4.0
min_gpa: 1.0
high_performers: 6028
=====
```

Hint: the lines inside are produced by `super().print_results()` — you only add the header and footer in the child class.

Task 4 — Association — Class Report (All Variants)

Create a `Report` class that uses `DataAnalyser` and `ResultSaver` through association (USES-A). `Report` does not inherit from either class — it holds references to them.

Technical requirements:

- ✓ Define class `Report`
- ✓ Define `__init__(self, analyser, saver)`:
 - Store `analyser` as `self.analyser` ← USES-A `DataAnalyser`
 - Store `saver` as `self.saver` ← USES-A `ResultSaver`
- ✓ Define method `generate(self)`:
 - Print "Generating report..."
 - Call `self.analyser.analyse()`
 - Call `self.analyser.print_results()`
 - Call `self.saver.save_json()`
 - Print "Report complete."

Console:

```
analyser = GpaAnalyser(students)
saver = ResultSaver(analyser.result, 'output/result.json')
report = Report(analyser, saver)
report.generate()
```

Generating report...

=====

GPA ANALYSIS REPORT

=====

total_students: 10000

average_gpa: 3.45

max_gpa: 4.0

min_gpa: 1.0

high_performers: 6028

=====

Result saved to output/result.json

Report complete.

Task 5

Task 5 — Polymorphism (All Variants)

Demonstrate polymorphism by creating multiple analyser objects of different types and calling the same method on each. The same call produces different behaviour depending on the object's class.

Technical requirements:

- ✓ Create at least 2 child class objects — use your own child class plus one more from a different variant
- ✓ Place them in a list called `analysers`
- ✓ Use a `for` loop to call `analyse()` and `print_results()` on each object
- ✓ Before the loop, print each object using `print(analyser)` to show `__str__`
- ✓ The loop must not know the type of each object — it just calls the same methods on all

Hint: you can create a small test list of 10 rows instead of loading the full CSV to keep the output short for the second analyser.

Console:

```
-----
Running all analysers:
-----

GpaAnalyser: GPA Statistics, 10000 students
=====
GPA ANALYSIS REPORT
=====
total_students: 10000
average_gpa: 3.45
max_gpa: 4.0
min_gpa: 1.0
high_performers: 6028
=====

CountryAnalyser: Country Analysis, 10000 students
=====
COUNTRY ANALYSIS REPORT
=====
total_students: 10000
total_countries: 15
top_3: [('USA', 856), ('India', 821), ('Canada', 803)]
=====
-----
```

Full program structure:

```
# Task 1 – base class
class DataAnalyser: ...

# Task 2 – child class
class GpaAnalyser(DataAnalyser): ...

# Task 3 – override print_results() inside child class

# Task 4 – association
```

```
class Report: ...

# main
fm = FileManager('students.csv')
fm.check_file()
fm.create_output_folder()

dl = DataLoader('students.csv')
dl.load()
dl.preview()

# Task 5 – polymorphism
analysers = [GpaAnalyser(dl.students), CountryAnalyser(dl.students)]
print('Running all analysers:')
for a in analysers:
    print(a)
    a.analyse()
    a.print_results()

# Task 4 – report
saver = ResultSaver(analysers[0].result, 'output/result.json')
report = Report(analysers[0], saver)
report.generate()
```